IIIT-Bangalore

Software Testing

CSE - 731

Professors:
Prof. Meenakshi D Souza

Submitted by:
Madhav Girdhar (IMT2022009)
Bhavya Kapadia (IMT2022095)

November 24, 2025

1 Source Code

- Repository Link : Software Testing

2 What is Mutation Testing

Mutation Testing is a software testing technique used both to create new test cases and to assess the effectiveness of existing ones. It involves making small, systematic modifications to a program's source code, known as *mutations*. These mutations simulate potential faults.
Each mutated version of the program is executed against the current test suite. If a test fails due to the mutation, the mutation is considered *killed*. If the tests still pass, the mutation is said to have *survived*. The overall strength and quality of the test suite can be measured by the proportion of mutations that are successfully killed.

3 Why Mutation Testing is Needed

Traditional coverage metrics—such as line, statement, or branch coverage—only indicate which parts of the code are executed during testing. However, they do not verify whether the tests are capable of detecting defects within that executed code. As a result, coverage alone can only reveal code segments that are certainly untested, but cannot ensure that the existing tests are effective at finding faults. Mutation testing addresses this gap by evaluating the fault-detection ability of the test suite.

4 Using PIT as a Mutation Testing Tool

PIT (PITest) is a widely used mutation testing framework for Java that integrates easily with build tools such as Maven and Gradle. It automatically generates mutants, runs the existing test suite against them, and reports which mutations were killed or survived. PIT is efficient, supports incremental analysis, and provides clear HTML reports that help developers identify weak or ineffective tests. Its ability to run quickly, even on large projects, makes it

a practical and powerful choice for improving test quality in real-world Java applications.

5 Objective of Mutation Testing

The main goal of mutation testing is to show how introduced mutations affect program behavior by successfully *killing* them. A mutant can be eliminated in two ways:

1. *Weak Mutant Killing*: For a mutant m in a program P that alters a specific location l, a test case t weakly kills m if the program state after executing t on P differs from the state obtained after executing t on the mutant m immediately following l.

2. *Strong Mutant Killing*: For a mutant m in a program P, a test case t strongly kills the mutant if the final output of P on t is different from the output produced by the mutant.

In this work, we adopt mutation testing with an emphasis on strong mutant killing. This approach provides a rigorous assessment of the code by ensuring that each mutation leads to a noticeable change in the program's final behavior, thereby enhancing the overall quality and reliability of the test suite.

6 Available Mutation Operators

A variety of mutation operators can be applied to introduce small, controlled faults into a program. Common operators include:

- Deleting entire statements.
- Duplicating or inserting statements, such as adding an unintended `goto fail;` line.
- Replacing boolean subexpressions with constant values `true` or `false`.
- Changing arithmetic operators, for example replacing `+` with `*`, or `-` with `/`.
- Modifying relational operators, such as switching `>` to $\geq$, `==`, or $\leq$.
- Substituting variables with others in the same scope, provided their types are compatible.
- Removing the entire body of a method.
- Altering logical operators, such as changing `&&` to `||` or vice versa.
- Negating conditional expressions, for instance converting `if (x > 0)` to `if (!(x > 0))`.

- Modifying return statements, such as replacing a return value with a default value (e.g., `0`, `null`, or `false`).
- Changing increment and decrement operators, such as replacing `i++` with `i--`.
- Altering loop boundaries or conditions, such as transforming `for (i < n)` into `for (i <= n)`.

For this project, we have incorporated several of these mutation operators. Due to time constraints, it is not feasible to include the complete set of available operators, so a selected subset has been used to demonstrate mutation testing effectively.

7 Mutation Testing Levels

Unit Mutation: Unit mutation targets small, individual components of a program, such as functions or methods. It involves modifying their internal logic—like changing operators or adjusting control flow—to evaluate whether the test suite can identify these localized changes. This ensures that each unit of the software behaves correctly in isolation.

Integration Mutation: Integration mutation, also known as interface mutation, focuses on the interactions between different components. Mutations are introduced at the connection points, often involving method calls. In this approach, both the caller and callee components are taken into account, allowing testers to verify the reliability and correctness of inter-component communication.

The Mutation Integration Operators covered are:

- IMCD : Integration Method Call deletion.
- IREM : Integration Return Expression Modification.
- IUOI : Integration Uniory Operator Insertion.

Detailed information about each file, the types of mutants generated, and whether those mutants were killed or survived is available in the PIT mutation testing report. This report is automatically generated in the `target` folder after compiling the project and running the mutation testing command. Some of Sample will look like this:

Mutations

14	1. removed conditional - replaced equality check with true → KILLED 2. removed conditional - replaced equality check with false → KILLED
15	1. removed conditional - replaced comparison check with true → KILLED 2. removed conditional - replaced comparison check with false → KILLED 3. changed conditional boundary → KILLED
18	1. removed conditional - replaced equality check with true → KILLED 2. removed conditional - replaced equality check with false → KILLED
19	1. removed call to org/example/model/CartItem::increment → KILLED
26	1. removed conditional - replaced equality check with false → SURVIVED 2. removed conditional - replaced equality check with true → KILLED
27	1. removed conditional - replaced comparison check with false → SURVIVED 2. removed conditional - replaced comparison check with true → KILLED 3. changed conditional boundary → SURVIVED
30	1. removed conditional - replaced equality check with false → KILLED 2. removed conditional - replaced equality check with true → KILLED
32	1. removed call to org/example/model/CartItem::decrement → KILLED
34	1. removed conditional - replaced equality check with true → KILLED 2. removed conditional - replaced equality check with false → KILLED
45	1. replaced double return with 0.0d for org/example/service/CartService::getSubtotal → KILLED
52	1. replaced return value with Collections.emptyMap for org/example/service/CartService::getItems → KILLED
56	1. removed call to java/util/Map::clear → KILLED

Figure 1: Mutants in Cart Service.

Mutations

14	1. removed conditional - replaced equality check with true → KILLED 2. removed conditional - replaced equality check with false → KILLED
15	1. removed conditional - replaced comparison check with true → KILLED 2. removed conditional - replaced comparison check with false → KILLED 3. changed conditional boundary → KILLED
18	1. removed conditional - replaced equality check with true → KILLED 2. removed conditional - replaced equality check with false → KILLED
19	1. removed call to org/example/model/CartItem::increment → KILLED
26	1. removed conditional - replaced equality check with false → SURVIVED 2. removed conditional - replaced equality check with true → KILLED
27	1. removed conditional - replaced comparison check with false → SURVIVED 2. removed conditional - replaced comparison check with true → KILLED 3. changed conditional boundary → SURVIVED
30	1. removed conditional - replaced equality check with false → KILLED 2. removed conditional - replaced equality check with true → KILLED
32	1. removed call to org/example/model/CartItem::decrement → KILLED
34	1. removed conditional - replaced equality check with true → KILLED 2. removed conditional - replaced equality check with false → KILLED
45	1. replaced double return with 0.0d for org/example/service/CartService::getSubtotal → KILLED
52	1. replaced return value with Collections.emptyMap for org/example/service/CartService::getItems → KILLED
56	1. removed call to java/util/Map::clear → KILLED

Figure 2: Mutants in Coupon Service.

8 Mutation Score

The mutation score represents the proportion of mutants that have been successfully killed out of the total mutants generated. It is calculated as:

$$\text{Mutation Score} = \frac{\text{Killed Mutants}}{\text{Total Mutants}} \times 100$$

A test suite is considered mutation adequate when it achieves a mutation score of 100%. Research has shown that mutation testing is a powerful technique for assessing the thoroughness and quality of test cases. However, its major drawback is the high computational cost associated with producing large numbers of mutants and executing the full test suite on each mutated version of the program.

9 Details of Source Code

Our project consists of multiple services, each responsible for a specific part of the business logic:

1. CartService – handles product addition, removal, quantity modification, and overall cart state management.
2. PriceService – computes item-wise and cart-level prices, including subtotal and dynamic recalculations.
3. CouponService – validates available coupons and applies eligible discount rules.
4. GSTService – calculates applicable GST based on the different product types.
5. Payment Strategy Service – supports multiple payment methods such as Card, UPI, and Cash, implemented using the Strategy Pattern.

Along with these, several utility and helper classes are implemented to support validations, record management, product modeling, and formatted receipt generation. Together, these components form a complete workflow from adding items to the cart all the way to generating the final bill.

These modules collectively involve a wide range of operators—including arithmetic operators (+, -, *, /), logical and comparison operators (¿, ¡, ==), conditional constructs, and iterative processes. Such elements naturally provide many potential mutation points, making the project a strong candidate for mutation testing.

Unlike simple algorithms, our shopping cart system represents a real-world workflow where accuracy of every calculation is critical. Even a small fault—such

as an incorrect subtotal, wrong tax percentage, misapplied discount, or wrong quantity update—can lead to significant errors in the final receipt. This makes mutation testing extremely valuable because it helps verify the robustness of our test cases against subtle logic changes.

10 Lines of Code

- Source Code : 888 lines.
- Test Code (Junit) : 1140 lines.
- Total Lines in Code : 2028.

11 Mutation Operators Used

1. **AOR** – Replaces arithmetic operators such as $+, -, *, /$ with alternatives.
2. **ROR** – Replaces relational operators like $>, <, >=, <=, ==, !=$.
3. **LCR** – Replaces logical connectors such as && and ||.
4. **AOD** – Deletes an arithmetic operator, e.g., $a + b$ becomes a.
5. **COD** – Removes conditional checks, forcing conditions to always be true or false.
6. **COI** – Inserts additional conditions or negations into existing conditional logic.
7. **ASR** – Replaces assignment operators like $+=, -=, *=, /=$ with alternatives.
8. **INC** – Mutates increment/decrement operations such as ++ and −−.
9. **UOI** – Inserts or inverts unary operators such as $-, +$, or logical negation !.
10. **RVR** – Replaces method return values with constants like 0, $null$, $true$, or $false$.

The results of the Mutation Testing process can be viewed in the generated HTML report located at `./target/pit-report/index.html`. This report provides detailed insights into the mutations applied and the corresponding test outcomes.

Pit Test Coverage Report

Project Summary

Number of Classes	Line Coverage		Mutation Coverage		Test Strength	
13	95%	163/172	87%	124/143	92%	124/135

Breakdown by Package

Name	Number of Classes	Line Coverage		Mutation Coverage		Test Strength	
org.example.model	3	93%	64/69	86%	42/49	88%	42/48
org.example.payment	6	100%	47/47	100%	39/39	100%	39/39
org.example.service	4	93%	52/56	78%	43/55	90%	43/48

Report generated by PIT 1.15.0

Enhanced functionality available at arcmutate.com

Figure 3: Overall Analysis.

Pit Test Coverage Report

Package Summary

org.example.model

Number of Classes	Line Coverage		Mutation Coverage		Test Strength	
3	93%	64/69	86%	42/49	88%	42/48

Breakdown by Class

Name	Line Coverage		Mutation Coverage		Test Strength	
CartItem.java	85%	17/20	71%	12/17	75%	12/16
Coupon.java	93%	26/28	89%	17/19	89%	17/19
Item.java	100%	21/21	100%	13/13	100%	13/13

Report generated by PIT 1.15.0

Figure 4: Analysis for model package.

Pit Test Coverage Report

Package Summary

org.example.service

Number of Classes	Line Coverage		Mutation Coverage		Test Strength	
4	93%	52/56	78%	43/55	90%	43/48

Breakdown by Class

Name	Line Coverage		Mutation Coverage		Test Strength	
CartService.java	100%	26/26	86%	18/21	86%	18/21
CouponService.java	69%	9/13	68%	17/25	94%	17/18
GSTService.java	100%	7/7	83%	5/6	83%	5/6
PriceService.java	100%	10/10	100%	3/3	100%	3/3

Report generated by PIT 1.15.0

Figure 5: Analysis for service package.

Pit Test Coverage Report

Package Summary

org.example.payment

Number of Classes	Line Coverage		Mutation Coverage		Test Strength	
6	100%	47/47	100%	39/39	100%	39/39

Breakdown by Class

Name	Line Coverage		Mutation Coverage		Test Strength	
CardPaymentService.java	100%	8/8	100%	10/10	100%	10/10
CashPaymentService.java	100%	4/4	100%	4/4	100%	4/4
PaymentReceipt.java	100%	16/16	100%	7/7	100%	7/7
PaymentResult.java	100%	9/9	100%	5/5	100%	5/5
PaymentService.java	100%	2/2	100%	4/4	100%	4/4
UPIPaymentService.java	100%	8/8	100%	9/9	100%	9/9

Report generated by PIT 1.15.0

Figure 6: Analysis for payment package.

12 Running the Project

To execute the project, follow the steps outlined below:

1. **Clone the repository:**
   `git clone https://github.com/madhav8511/SoftwareTesting.git`
2. **Navigate to the project directory:**
   `cd SoftwareTesting`
3. **Compile the project using Maven:**
   `mvn clean install`
4. **Run all JUnit test cases:**
   `mvn test`
5. **Perform Mutation Testing using PIT:**
   `mvn org.pitest:pitest-maven:mutationCoverage`
6. **Run the main application:**
   `mvn exec:java -Dexec.mainClass="org.example.Main"`

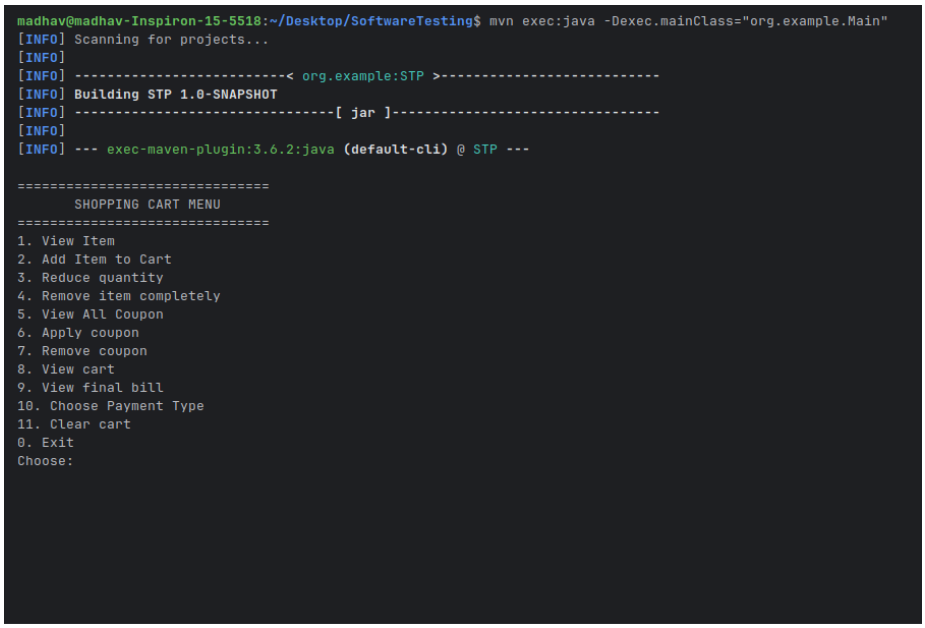

Figure 7: Project View.

13 Team Contributions

The implementation of the shopping cart logic—including item management, price calculation, tax computation, and coupon discount application—was collaboratively developed by both team members. All debugging, testing activities, and the preparation of the final project report were also carried out jointly, ensuring equal contribution throughout the project.

- **Madhav Girdhar** : Project Setup, Junit TestCases for Models and Payments, Comments in Code.
- **Bhavya Kapadia** : Project Setup, Junit TestCases for Service and Payments, Comments in Code.